Machine Learning in Time Series

Lecture 5: Trees, Boosting, and Ensembles

Instructor: Marcelo Villena, PhD

Spring 2026

PhD Seminar in Econometrics / Data Science
Università degli Studi di Bari Aldo Moro

Agenda

1. Seminar Overview & Roadmap
2. From Classical Non-Linearity to Trees
3. Ensembles Part 1: Bagging & Random Forests
4. Ensembles Part 2: Gradient Boosting
5. Readings Discussion & Interpretability
6. Lecture 5 Mini-Lab: The Ensemble Horse Race
7. Visualizing the Horse Race
8. Residual Diagnostics
9. Discussion & Synthesis
10. Concluding Remarks

Seminar Overview & Roadmap

The 6-Lecture Journey

Where we are

In Lecture 4, we saw that Deep Learning requires massive datasets to overcome variance. Today, we focus on **Ensemble Methods**. These algorithms dominate tabular, non-linear forecasting because they capture complex interactions while remaining robust on moderate sample sizes.

Lecture 1: Historical Evolution, the Prediction Problem, and Shrinkage.

Lecture 2: Kernel Methods and Support Vector Machines (SVM).

Lecture 3: High-Dimensional Linear Dynamics (BVAR, Sparse VAR).

Lecture 4: Deep Learning for Sequences (RNN/LSTM).

Lecture 5: Trees, Boosting, and Ensembles.

Lecture 6: NLP, Attention, and Causal ML.

From Classical Non-Linearity to Trees

The Econometric Motivation

Macroeconomic dynamics are rarely perfectly linear. Asymmetries and threshold effects are everywhere.

Classical Approaches vs. Machine Learning

- **Classical Econometrics:** We manually specify regime-switching models (e.g., Threshold Autoregression [TAR] or Markov-Switching VARs). *The problem:* We have to guess the threshold variable in advance (e.g., "Is GDP growth < 0?").
- **Machine Learning (Trees):** The algorithm searches purely for the optimal, data-driven thresholds across hundreds of variables simultaneously to partition the data space.

Classification and Regression Trees (CART) - The Intuition

Instead of fitting a single global line (OLS) or relying on massive matrix gradients (Neural Nets), a decision tree breaks the prediction problem into a sequence of simple "If/Then" questions.

The Visual Intuition: Imagine a 2D scatterplot with Unemployment on the X-axis and Interest Rates on the Y-axis. The CART algorithm literally draws straight, orthogonal lines (cuts) through the graph, slicing it into distinct rectangular boxes. Every historical observation inside a specific box is averaged together to create the forecast for that region.

The Mathematics of the Split

How does it decide where to draw the line?

The algorithm searches across all predictors j and all possible cutpoints s to split the predictor space into two regions (R_1, R_2) that minimize the Residual Sum of Squares (RSS):

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

- j = The variable chosen for the split (e.g., Unemployment Lag 1).
- s = The exact threshold (e.g., $u_{t-1} > 5.5\%$).
- c_1, c_2 = The constant forecast assigned to that region (the mean).

The Fatal Flaw of a Single Tree

The Pros:

- Automatically captures interactions (e.g., "If rates are high AND unemployment is rising").
- Invariant to monotonic transformations (no scaling required!).
- Can be robust to missingness depending on implementation (some support native missing-value handling; otherwise imputation is required).

The Cons:

- **Extreme Variance.** If you change the training data by just 5%, the first split might change entirely, leading to a completely different tree topology.
- A single, deep tree will relentlessly overfit the training set (low bias, high variance, especially for deep trees).

The Solution: We never use single trees in modern forecasting. We use Ensembles.

Ensembles Part 1: Bagging & Random Forests

Bootstrap Aggregating (Bagging)

How do we fix the high variance of a single tree? We use the statistical power of averaging.

The Bagging Algorithm

1. Draw B different bootstrap samples from the training data (with replacement).
2. Train a deep, unpruned tree on each sample.
3. Average the predictions of all B trees: $\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$

The Math: If we average predictions from B correlated trees, the variance of their mean is roughly:

$$\mathrm{Var}(\bar{f}) \approx \rho\sigma^2 + (1-\rho)\sigma^2/B$$

By averaging hundreds of trees, the $(1-\rho)\sigma^2/B$ term shrinks to zero. However, the correlation penalty ($\rho\sigma^2$) remains. **Random Forests reduce ρ by forcing decorrelated splits.**

Random Forests (Breiman, 2001) - The Intuition

Problem in macro forecasting with persistent predictors: If Lag 1 dominates, bootstrapped trees remain highly correlated.

The "Wisdom of Crowds" Fix (Feature Randomness): Each time a split is considered in a Random Forest, the algorithm is forced to choose from a small, random sample of m predictors (where $m \approx \sqrt{p}$). It forces the algorithm to explore "weaker" macroeconomic variables, structurally decorrelating the trees and creating a truly wise crowd!

Ensembles Part 2: Gradient Boosting

The Boosting Philosophy - The "Golf" Intuition

Random Forests build deep, independent trees in *parallel* to reduce variance.
Boosting builds shallow, dependent trees *sequentially* to reduce bias.

The Golf Analogy: Imagine trying to hit a golf ball into a hole.

- Your first swing (Tree 1) gets you to the green. You missed by 10 feet.
- Your second swing (Tree 2) doesn't aim for the hole from the tee—it aims to fix the exactly 10-foot *error* (the residual) of the first swing.
- Your third swing (Tree 3) fixes the remaining 2-inch error of the second swing.

The Mathematics of Boosting

Instead of fitting trees to the target y, Boosting fits each new tree to the **residuals** of the previous ensemble.

The Sequential Update Rule:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$$

Where λ is the **learning rate** (shrinkage parameter). We slowly add small, incremental updates to our forecast. If a time-series observation is consistently mispredicted, the algorithm forces subsequent trees to focus strictly on that specific error.

Modern Implementations: XGBoost

The dominant algorithm in tabular Kaggle competitions and modern macro-forecasting is **Extreme Gradient Boosting (XGBoost)**.

Why is XGBoost so powerful? Unlike traditional boosting, XGBoost explicitly adds econometric **Regularization** directly into the loss function being minimized at each tree split:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$$

It penalizes the number of leaves (T) and adds an L_2 Ridge penalty (λ) to the leaf weights. It actively prevents the boosting sequence from memorizing macroeconomic noise.

Readings Discussion & Interpretability

Reading: Goulet Coulombe et al. (2022)

How is Machine Learning Useful for Macroeconomic Forecasting?

The authors conduct a massive horse race of ML models across hundreds of macroeconomic variables. Their central finding: **Random Forests and Boosting dominate.**

Key Takeaways:

- Neural networks struggle because the signal-to-noise ratio in macroeconomics is too low for deep architectures.
- Trees excel because they naturally handle the *non-linearities* and *regime-dependencies* inherent in business cycles (e.g., the relationship between housing starts and GDP changes during a recession).
- *Warning:* They are strong interpolators but weak extrapolators: outside-support forecasts are unreliable: trees partition observed regions and typically do not extrapolate smoothly beyond the training range.

Interpretability: Opening the Ensemble Black Box

How do we extract economic inference from a forest of 500 trees?

1. Partial Dependence Plots (PDPs): PDPs marginalize over all other features to show the isolated, non-linear effect of one variable on the forecast. (e.g., "Holding all else equal, how does changing interest rates alter the forecast trajectory?").

2. SHAP Values (Shapley Additive exPlanations): Uses cooperative game theory. Think of the predictors as players on a sports team. SHAP calculates exactly how many "points" (forecast percentage points) each specific macroeconomic lag contributed to the final prediction on a specific date.

Lecture 5 Mini-Lab: The Ensemble Horse Race

Mini-Lab: Trees vs. Boosting vs. Regularization

The Task

We return to our Spine Project (YoY Core CPI, AR(12) lags). Today, we run a rigorous OOS horse race between Ridge Regression, a Random Forest, and Gradient Boosting (XGBoost).

Pipeline Requirements:

1. **Data Prep:** No 3D tensors required today! We return to standard 2D feature matrices. (Scaling is optional for trees, but required for Ridge).
2. **Random Forest:** Use the `ranger` package. Set $B = 500$ trees.
3. **XGBoost:** Use the `xgboost` package. Tuning the learning rate (η) and max tree depth is critical to prevent overfitting. **Tune with blocked/rolling validation (time-slice CV).** Baseline lab uses a fixed holdout OOS; extension: rolling refit evaluation.
4. **Deliverable:** Produce a **Model Card** summarizing the OOS RMSE and the chosen hyperparameters for the winning model.

The Model Card: Accuracy vs. Compute

The following table summarizes the OOS performance on YoY Core CPI using a strict temporal split.

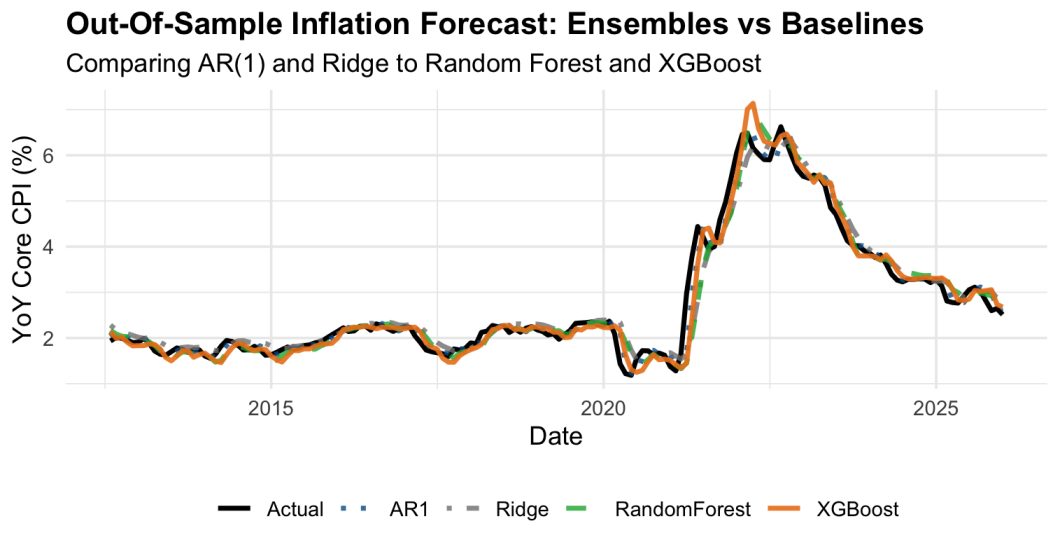
Model	OOS RMSE	Fit (sec)	Pred (sec)	Total Time
AR(1) Baseline	**0.2161**	0.001	0.001	**0.002s**
XGBoost	0.2757	0.062	0.001	0.063s
Random Forest	0.3246	0.185	0.008	0.193s
Ridge Regression	0.3285	0.016	0.001	0.017s

The "Humility Check" Result

The simplest model (AR1) outperformed the most complex ensembles. This occurs because inflation is highly persistent ($y_t \approx y_{t-1}$); with limited turning-point examples, higher-capacity models can overfit idiosyncratic shocks and lose OOS performance.

Visualizing the Horse Race

Graph 1: Forecast Path Horse Race



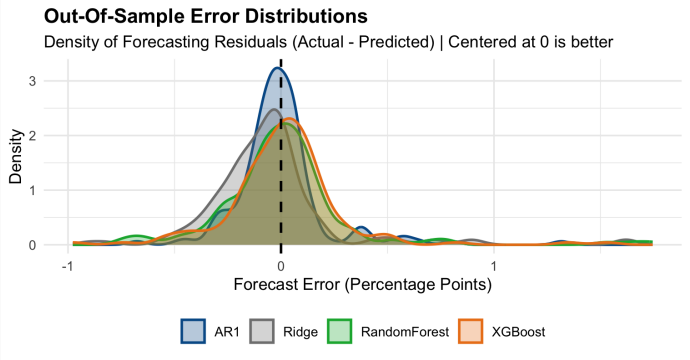
Graph 1: Forecast Path Horse Race

Key Observations:

- **Persistence:** Notice how **AR(1)** (dotted blue) hugs the actual path most closely during stable periods.
- **Turning Points:** **XGBoost** captures the post-2020 acceleration better than Random Forest due to its regularization and shallow tree structure.
- **Extrapolation:** All models struggled with the unprecedented 2022 peak, but linear models extrapolated the trend further than tree-based ensembles.

Residual Diagnostics

Graph 2: Out-Of-Sample Error Distributions



Interpreting the Density:

- **Tightness:** The **AR(1)** distribution is the "tallest" and most centered at 0, indicating lower overall variance in errors.
- **Skewness:** Notice the right-hand tails. All models exhibit a degree of under-prediction during the high-inflation regime.
- **The RF Tail:** The **Random Forest** has a wider base, reflecting higher OOS variance caused by deeper, unpruned trees.

Discussion & Synthesis

Why didn't ML win?

1. **Signal-to-Noise Ratio:** In macroeconomics, the signal is often simple (persistence). Advanced models have a high capacity to "memorize" idiosyncratic historical shocks that do not repeat OOS.
2. **The Data Hunger of Trees:** Random Forests and XGBoost require many "examples" of regime shifts to learn them. In a sample of moderate size, we only have a handful of major turning points.
3. **Regularization is King:** XGBoost performed better than RF/Ridge because we manually restricted its depth ($max_depth = 3$). **Structural restraint beats algorithmic complexity** in small-sample time series.

Lab Conclusion

The AR(1) is your "Hard Baseline." If your ML model cannot beat an AR(1), it is likely overfitting.

Concluding Remarks

Project: 10-Minute Check-In

Our Standard Closing Questions

1. **What changed in OOS RMSE?**
2. **Which failure mode did we fix (or create)?**

Today's Expected Answers:

- **RMSE Shift:** In this sample, AR(1) is a hard baseline and can beat ensembles; RF/XGBoost may help mainly around localized nonlinearities but are not guaranteed to win in small macro samples.
- **Failure Mode Fixed:** Nonlinear interactions can be modeled without specifying thresholds by hand.
- **Failure Mode Created:** Extrapolation and sensitivity to hyperparameters; limited turning points can lead to overfitting.

Looking Ahead: Lecture 6 (The Finale)

Next Session: NLP, Attention, and Causal ML

In our final lecture, we push to the absolute frontier of modern econometrics. How do we extract numerical data from central bank speeches? And how do we do causal inference with Machine Learning?

Topics to prepare for:

- Natural Language Processing (NLP) for economics.
- The logic of Double/Debiased Machine Learning (DML) for causal inference.
- Final Project Pitches!

Reminder: Reading memos for Chernozhukov et al. (2018) and Athey et al. (2021) are due before the final Lecture!